

Bloque 06_active_directory

Índice de módulos

- AD CS y certificados abusables
- Como se ataca Active Directory
- Pass-the-Ticket, ACL y trusts en Active Directory
- AS-REP roasting y Kerberoasting
- BloodHound: el grafo de permisos
- Kerberos en dos minutos
- Enumeracion de LDAP
- Hashes NTLM y Pass-the-Hash

AD CS y certificados abusables

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

AD CS emite certificados para usuarios y equipos. Una plantilla mal configurada puede permitir a un usuario solicitar un certificado con identidad privilegiada y autenticarse como ella.

La CA confía en los atributos de la solicitud y en los permisos de la plantilla; combinaciones como enrollment abierto y sujeto controlable convierten una mala configuración en escalada de dominio.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando el dominio expone una CA y tienes una cuenta válida, especialmente si BloodHound o certipy muestran plantillas editables o enrolamiento amplio.

Cómo identificarla

- Certificate Templates
- ENROLLEE_SUPPLIES_SUBJECT
- Client Authentication
- ESC1
- CA web enrollment

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Enumera CAs y plantillas, identifica una plantilla vulnerable y solicita un certificado de prueba con una identidad permitida.

Evidencia esperada: Conoces la plantilla, los permisos que la hacen vulnerable y si el certificado permite autenticación como la identidad objetivo.

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
certipy find -u '$USER@$DOMAIN' -p '$PASS' -dc-ip $DC_HOST -vulnerable
```

Objetivo y contexto: Enumera plantillas AD CS y marca configuraciones potencialmente vulnerables.

Resultado esperado: CA, plantilla, permisos y una razón concreta de vulnerabilidad.

Capa 2: prueba documentada

```
certipy req -u '$USER@$DOMAIN' -p '$PASS' -ca '$CA_NAME' -template '$TEMPLATE' -upn administrator@$DOMAIN -dc-ip $DC_HOST
```

Objetivo y contexto: Solicita un certificado de laboratorio solo cuando la plantilla y el alcance lo permiten.

Resultado esperado: Certificado .pfx emitido o un rechazo explicable.

Anatomía de los payloads

La primera prueba es `certipy find -u '$USER@$DOMAIN' -p '$PASS' -dc-ip $DC_HOST -vulnerable`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Conoces la plantilla, los permisos que la hacen vulnerable y si el certificado permite autenticación como la identidad objetivo.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Enumera CA y plantillas.
- Revisa permisos y EKUs.
- Solicita un certificado controlado y valida la autenticación.
- Documenta la causa raíz y revoca material de prueba.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No hay CA AD CS publicada.
- La plantilla requiere aprobación manual y no tienes ese flujo autorizado.

- El alcance excluye pruebas de certificados.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Detectar plantillas de Active Directory Certificate Services que permiten suplantar identidades. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.

4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (ad-cs)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Como se ataca Active Directory

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Active Directory es el directorio que gestiona usuarios, equipos y permisos de una red Windows. Atacarlo no va de un exploit puntual, sino de conseguir una identidad (un usuario) y usar sus permisos para conseguir otra mas fuerte, repitiendo hasta llegar a Domain Admin o al hash de krbtgt.

AD esta hecho de relaciones de confianza (grupos, delegaciones, ACLs, sesiones). Cada credencial abre nuevas relaciones que explotar. Por eso la mentalidad es distinta: importa mas el grafo de permisos (bloodhound lo mapea) que la version de un servicio.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cuanto el recon confirme un DC (puertos 88 Kerberos + 389 LDAP + 445). El trabajo previo imprescindible es resolver nombres y hora, o Kerberos falla con errores confusos.

Cómo identificarla

- Puertos 88, 389, 445, 636, 3268 abiertos en el mismo host.
- nmap/enum filtra un nombre de dominio (algo.local) y el hostname del DC.
- Errores de 'clock skew' al usar herramientas: tu reloj no cuadra con el DC.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Escanea los puertos tipicos de DC y confirma que 88 (Kerberos) responde.

Evidencia esperada: 88+389+445 abiertos en el mismo host, y los scripts de nmap/enum filtran el nombre del dominio.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
nmap -sC -sV -Pn -p 53,88,135,139,389,445,464,636,3268,5985 $IP
```

Objetivo y contexto: Escanea los puertos típicos de un DC de una vez. La combinación 88+389+445 confirma que estas ante un dominio y no una máquina suelta.

Resultado esperado: Puerto 88 abierto = DC. Los scripts LDAP/SMB filtran el nombre del dominio y del DC: anotalos para /etc/hosts antes de seguir.

Anatomía de los payloads

La primera prueba es `nmap -sC -sV -Pn -p 53,88,135,139,389,445,464,636,3268,5985 $IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

88+389+445 abiertos en el mismo host, y los scripts de `nmap/enum` filtran el nombre del dominio.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Prepara nombres (/etc/hosts) y sincroniza la hora con el DC.
- Consigue el primer usuario: `enum de SMB (smb-enum)`, `rid-brute` y `password spraying` controlado.
- Saca hashes crackeables por Kerberos (`asrep-kерberoast`) y mapea permisos con `bloodhound`.
- Extrae hashes del objetivo y autentica con `ntlm-pth` hasta Domain Admin.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El objetivo es una máquina Windows standalone sin dominio (no hay 88/389 abiertos).
- No tienes ninguna vía de conseguir un primer usuario (sin acceso anónimo, sin usuarios enumerables).
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

En AD no explotas una maquina: transformas una identidad en mas permisos, en cadena, hasta Domain Admin. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (ad-modelo)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Pass-the-Ticket, ACL y trusts en Active Directory

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Pass-the-Ticket reutiliza un TGT o TGS válido sin conocer la password. Las ACL conceden acciones sobre objetos del dominio y los trusts conectan límites de autenticación entre dominios o bosques.

AD decide acceso por identidades, tickets y permisos encadenados. Un ticket robado o una ACL como GenericAll, WriteDACL o ForceChangePassword puede abrir una ruta que no aparece al mirar solo grupos directos.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando obtienes un ccache/kirbi, BloodHound muestra una arista abusada o el dominio mantiene trusts con otros dominios.

Cómo identificarla

- ccache
- kirbi
- KRB5CCNAME
- GenericAll
- WriteDACL
- ForceChangePassword
- TrustedDomain

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Comprueba identidad, realm, validez temporal y servicio accesible con Kerberos; valida cada ACL o trust antes de ejecutar cambios.

Evidencia esperada: Una ruta de identidad a recurso o privilegio con precondiciones, limite de alcance e impacto demostrable.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
impacket-getTGT $DOMAIN/$USER:$PASS -dc-ip $IP
```

Objetivo y contexto: Obtiene un TGT de laboratorio y crea el ccache que usaras para validar autenticacion por ticket.

Resultado esperado: Fichero ccache y periodo de validez del ticket.

Capa 2: prueba documentada

```
export KRB5CCNAME=$USER.ccache; impacket-psexec -k -no-pass $DOMAIN/$USER@$DC_FQDN
```

Objetivo y contexto: Demuestra autenticacion Kerberos con ticket sin volver a enviar la password.

Resultado esperado: Acceso autorizado o error de realm, DNS, hora o permisos que debes interpretar.

Anatomía de los payloads

La primera prueba es `impacket-getTGT $DOMAIN/$USER:$PASS -dc-ip $IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una ruta de identidad a recurso o privilegio con precondiciones, limite de alcance e impacto demostrable.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma realm, DNS y hora.
- Inspecciona tickets y permisos.
- Selecciona una accion minima.
- Valida acceso Kerberos sin password.
- Documenta la cadena completa.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El ticket esta expirado, pertenece a otro realm o el reloj no esta sincronizado.
- La arista de BloodHound no concede una accion util sobre el objetivo.
- El dominio confiado queda fuera de alcance.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Transformar tickets, permisos de objetos y relaciones de confianza en rutas de movimiento lateral explicables. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (ad-tickets-trusts)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

AS-REP roasting y Kerberoasting

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Son dos ataques que piden tickets de kerberos y se quedan con la parte cifrada para crackearla offline. AS-REP roasting funciona contra usuarios con la preautenticación desactivada y no necesita credenciales. Kerberoasting pide tickets de cuentas de servicio (con SPN) y requiere una credencial de dominio cualquiera.

El ticket va cifrado con una clave derivada de la password del usuario o de la cuenta de servicio. Si esa password es débil, la crackeas offline sin tocar el DC (sin riesgo de bloqueo). Las cuentas de servicio suelen tener passwords viejas y privilegios altos: premio doble.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

AS-REP: en cuanto tengas una lista de usuarios (smb-enum), aunque no tengas password. Kerberoast: en cuanto consigas UNA credencial válida. Ambos encajan en la fase AD (ad-modelo).

Cómo identificarla

- GetNPUsers devuelve hashes \$krb5asrep\$ (hay usuarios sin preauth).
- GetUserSPNs lista cuentas de servicio con SPN y devuelve \$krb5tgs\$.
- Los nombres de esas cuentas huelen a privilegio (svc_sql, backup, admin).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Lanza GetNPUsers -no-pass (AS-REP) o GetUserSPNs -request con una credencial (Kerberoast).

Evidencia esperada: Un hash \$krb5asrep\$ o \$krb5tgs\$ en la salida. Sin hash, ese vector concreto no aplica en este dominio.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
impacket-GetNPUsers $DOMAIN/ -usersfile users.txt -no-pass -dc-ip $IP
```

Objetivo y contexto: AS-REP roasting sin credenciales: pide tickets para usuarios con preauth desactivada. Solo necesitas la lista de usuarios y la IP del DC.

Resultado esperado: Hashes \$krb5asrep\$ para los usuarios vulnerables. Cada uno es crackeable offline (hashcat -m 18200). Sin salida = nadie tiene preauth desactivada.

Capa 2: prueba documentada

```
impacket-GetUserSPNs $DOMAIN/$USER:$PASS -dc-ip $IP -request
```

Objetivo y contexto: Kerberoasting: con una credencial de dominio pides los tickets de las cuentas de servicio. Esas cuentas suelen tener passwords debiles y muchos permisos.

Resultado esperado: Hashes \$krb5tgs\$ junto al nombre del servicio. Prioriza los que parezcan admin; crackea con hashcat -m 13100.

Anatomía de los payloads

La primera prueba es `impacket-GetNPUsers $DOMAIN/ -usersfile users.txt -no-pass -dc-ip $IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un hash \$krb5asrep\$ o \$krb5tgs\$ en la salida. Sin hash, ese vector concreto no aplica en este dominio.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- AS-REP: pasa la lista de usuarios a GetNPUsers con -no-pass.
- Kerberoast: con una credencial, pide los tickets de SPN con GetUserSPNs -request.
- Crackea los hashes con hashcat (modo 18200 AS-REP, 13100 Kerberoast) y rockyou.
- Cada password crackeada -> nueva identidad: vuelve a enumerar el dominio con ella.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- AS-REP: ningún usuario tiene la preautenticación desactivada (GetNPUsers no devuelve nada).

- Kerberoast: no hay cuentas de servicio con SPN registrado en el dominio.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Dos formas de sacar hashes crackeables de Kerberos: sin credenciales (AS-REP) o con una cualquiera (Kerberoast). La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (asrep-kerberoast)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

BloodHound: el grafo de permisos

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

BloodHound recolecta usuarios, grupos, equipos, sesiones y ACLs del dominio y los representa como un grafo. En lugar de adivinar que permiso abusar, ves la ruta concreta: quien puede resetear la password de quien, que grupo tiene GenericAll sobre otro, donde hay sesiones de admin que robar.

En un dominio real las relaciones de permisos son miles y a mano es imposible. El grafo revela caminos no obvios (encadenar tres permisos mediocres para llegar a DA) que definen el ataque en AD (ad-modelo).

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cuanto tengas UNA credencial valida. Es lo que haces cuando el roasting (asrep-kerberoast) no basto o para planificar el salto final a Domain Admin.

Cómo identificarla

- Aristas 'GenericAll', 'WriteDACL', 'ForceChangePassword' hacia grupos privilegiados.
- Sesiones de un admin en una maquina donde tu tienes acceso.
- Un camino corto marcado hacia el grupo Domain Admins.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Corre bloodhound-python con la credencial y confirma que el .zip se genera sin errores de autenticacion.

Evidencia esperada: El .zip se importa en la GUI y aparece tu usuario en el grafo; 'Shortest paths to Domain Admins' muestra al menos una ruta.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
bloodhound-python -d $DOMAIN -u $USER -p $PASS -c all -ns $IP --zip
```

Objetivo y contexto: Recolecta todo el grafo del dominio con una sola credencial, sin necesidad de pisar una maquina Windows. -c all junta toda la informacion; --zip lo deja listo para importar.

Resultado esperado: Un .zip para arrastrar a la GUI de BloodHound. Allí marca tu usuario como owned y lanza la consulta de rutas a Domain Admins.

Anatomía de los payloads

La primera prueba es `bloodhound-python -d $DOMAIN -u $USER -p $PASS -c all -ns $IP --zip`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El .zip se importa en la GUI y aparece tu usuario en el grafo; 'Shortest paths to Domain Admins' muestra al menos una ruta.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Recolecta los datos del dominio con un colector (bloodhound-python o SharpHound).
- Importa el zip en la GUI y marca tu usuario como 'owned'.
- Usa 'Shortest paths to Domain Admins' y estudia cada arista del camino.
- Abusa del permiso concreto (cada tipo de arista tiene su tecnica) y avanza un paso.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No tienes ninguna credencial de dominio todavía: BloodHound necesita autenticarse para recolectar.
- El dominio es trivialmente pequeño (pocos usuarios/equipos) y ya ves el camino a mano.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Mapea el dominio como un grafo y te muestra la ruta mas corta de tu usuario a Domain Admin. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (bloodhound)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Kerberos en dos minutos

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Kerberos es como AD autentica sin mandar la password por la red. Al iniciar sesión, el usuario obtiene un TGT (ticket para pedir tickets) del DC. Luego, para usar un servicio, cambia el TGT por un TGS (ticket de servicio). Todo gira en torno a tickets cifrados con claves derivadas de passwords.

Porque esos tickets van cifrados con la clave del usuario o del servicio, y se pueden pedir en situaciones que permiten crackearlos offline: eso es lo que explotan asrep-kerberoast. Además, si robas o falsificas tickets, te haces pasar por otro sin su password.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Es teoría de apoyo para la fase AD (ad-modelo). No hace falta dominarla para atacar, pero entender TGT/TGS te dice por qué funcionan los ataques de roasting y de tickets.

Cómo identificarla

- Puerto 88 abierto: hay Kerberos y por tanto un DC.
- Errores 'KRB_AP_ERR_SKEW' = tu reloj está desincronizado con el DC.
- Hashes que empiezan por \$krb5asrep\$ o \$krb5tgt\$ al hacer roasting.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Sincroniza tu reloj con el DC si ves KRB_AP_ERR_SKEW en cualquier herramienta de Kerberos.

Evidencia esperada: El error de clock skew desaparece y las herramientas de impacket dejan de fallar por tiempo.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
sudo ntpdate $DC_HOST
```

Objetivo y contexto: Sincroniza tu reloj con el DC. Kerberos rechaza tickets si el desfase supera unos minutos; este es el arreglo del clásico error de clock skew.

Resultado esperado: Imprime el ajuste de tiempo aplicado. Tras esto, las herramientas de Kerberos (impacket) dejan de fallar por skew. Alternativa: faketime.

Anatomía de los payloads

La primera prueba es `sudo ntpdate $DC_HOST`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El error de clock skew desaparece y las herramientas de impacket dejan de fallar por tiempo.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Asegura nombres y hora: Kerberos es muy sensible al reloj (max ~5 min de desfase).
- Con o sin credencial, intenta roasting para sacar tickets crackeables (asrep-kerberoast).
- Crackea los tickets offline para obtener nuevas credenciales.
- Con credenciales de mas nivel, sigue el grafo de permisos hacia Domain Admin.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- No es un entorno AD (no hay puerto 88): Kerberos no aplica.
- Tu reloj y el del DC estan sincronizados y no ves errores de skew: no hace falta tocar nada aqui.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El sistema de tickets de AD; entender TGT y TGS explica por que existen AS-REP roast y Kerberoasting. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (kerberos)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Enumeracion de LDAP

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

LDAP (puerto 389, o 636 cifrado) es el protocolo que Active Directory (y otros directorios) usan para almacenar y consultar usuarios, grupos, equipos y su estructura organizativa. Muchos servidores permiten un 'bind anonimo' (conectar sin credenciales) que, aunque limitado, a veces basta para leer el naming context y enumerar usuarios.

El bind anonimo existe para operaciones basicas de consulta sin necesitar cuenta, pero si el administrador no lo restringe, expone la estructura del directorio (nombres de usuario, grupos, a veces atributos sensibles) a cualquiera que se conecte, sin autenticar. Es el equivalente en AD de la sesion nula de smb-enum.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando el recon confirma un DC (puerto 389/636 abierto junto a 88 Kerberos y 445 SMB). Es parte del arranque de la fase AD: mira tambien ad-modelo.

Cómo identificarla

- Puertos 389/636 abiertos junto a 88 (Kerberos) y 445 (SMB): confirma que es un DC.
- Ldapsearch sin credenciales (bind anonimo) devuelve resultados en vez de error de autenticacion.
- El naming context revela el nombre del dominio en formato DC=...

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Intenta un bind anonimo y pide el naming context base con ldapsearch.

Evidencia esperada: Una respuesta con el defaultNamingContext (algo como DC=dominio,DC=local) confirma que el bind anonimo funciona y puedes consultar el directorio.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
ldapsearch -x -H ldap://$IP -s base namingcontexts
```

Objetivo y contexto: Bind anonimo (-x sin credenciales) pidiendo el naming context base. Es la prueba mínima: si responde, el bind anonimo esta permitido.

Resultado esperado: Un defaultNamingContext tipo DC=corp,DC=local. Confirma el nombre del dominio y que puedes seguir consultando sin credenciales.

Capa 2: prueba documentada

```
ldapsearch -x -H ldap://$IP -D '' -w '' -b 'DC=corp,DC=local' '(objectClass=user)'
```

Objetivo y contexto: Con el naming context ya conocido, pide todos los objetos de tipo usuario via bind anonimo. Es la enumeracion completa de usuarios sin credenciales.

Resultado esperado: Una lista de usuarios del dominio con sus atributos visibles. Extrae los nombres para spraying o Kerberoasting.

Anatomía de los payloads

La primera prueba es `ldapsearch -x -H ldap://$IP -s base namingcontexts`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una respuesta con el defaultNamingContext (algo como DC=dominio,DC=local) confirma que el bind anonimo funciona y puedes consultar el directorio.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Prueba el bind anonimo pidiendo el naming context base; te confirma si esta permitido y te da el nombre del dominio.
- Si el bind anonimo funciona, enumera usuarios y grupos con una consulta mas amplia (base DN completo).
- Con una credencial (aunque sea debil), repite la consulta: el bind autenticado suele revelar mucho mas que el anonimo.
- Los usuarios que saques alimentan spraying y roasting: mira `asrep-kberoast`.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El servidor rechaza el bind anonimo (necesitas credenciales validas para cualquier consulta).
- No es un entorno AD/LDAP: el puerto 389 no responde o pertenece a otro servicio.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

LDAP guarda el directorio de usuarios y grupos de un dominio; un bind anonimo te deja consultarlo sin credenciales. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(ldap-enum\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.

Hashes NTLM y Pass-the-Hash

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Windows guarda las passwords como hashes NTLM. Para muchos protocolos de autenticación, el hash ES la credencial: si lo tienes, puedes autenticarte SIN conocer la password en claro. Eso es Pass-the-Hash (PtH), y convierte un volcado de hashes directamente en acceso.

El protocolo NTLM usa el hash como secreto compartido, no la password. Por eso volcar hashes (con secretdump/DCSync tras conseguir privilegios) permite moverte por la red autenticandote con el hash. Solo necesitas crackear si el servicio exige la password en claro o quieres reutilizarla en sitios no-NTLM.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ya has volcado hashes (por admin local, DCSync o del SAM) en la fase AD (ad-modelo). Es la forma habitual de usar el hash de administrator o de una cuenta de servicio sin perder tiempo crackeando.

Cómo identificarla

- secretdump te da líneas usuario:rid:lmhash:nthash.
- nxc marca (Pwn3d!) al probar el hash: tienes ejecución.
- Tienes el NT hash de administrator o krbtgt.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

nxc smb \$!P -u administrator -H <NTLM> y revisa si marca (Pwn3d!).

Evidencia esperada: (Pwn3d!) en la salida de nxc, o un login exitoso por evil-winrm -H.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
impacket-secretsdump $DOMAIN/$USER:$PASS@$IP
```

Objetivo y contexto: Con una cuenta con privilegios (admin local o derechos de replicacion/DCSync) vuelca los hashes NTLM, incluidos administrator y krbtgt.

Resultado esperado: Lineas usuario:rid:lm:nt. El NT hash de administrator sirve para Pth; el de krbtgt para Golden Ticket. Copia los que apunten a cuentas privilegiadas.

Capa 2: prueba documentada

```
evil-winrm -i $IP -u administrator -H <NTLM>
```

Objetivo y contexto: Pass-the-Hash por WinRM: te autenticas con el hash NTLM (-H) sin la password. Si el crackeo falla o tarda, esto te da la shell igual.

Resultado esperado: Un prompt PS de administrator (whoami lo confirma). Desde ahí recoge la flag y documenta la cadena que te lleve hasta el hash.

Anatomía de los payloads

La primera prueba es `impacket-secretsdump $DOMAIN/$USER:$PASS@$IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

(Pwn3d!) en la salida de `nxc`, o un login exitoso por `evil-winrm -H`.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Vuelca los hashes del objetivo (secretsdump con una cuenta con privilegios).
- Prueba el NT hash contra SMB/WinRM con -H en vez de -p.
- Si (Pwn3d!), entra con `evil-winrm -H` para una shell interactiva.
- El hash de krbtgt permite Golden Ticket (persistencia total del dominio).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El servicio exige la password en claro y no acepta autentificación NTLM (algunos servicios/API la desactivan).
- No tienes ningún hash volcado todavía: esto es un paso posterior a `secretsdump`, no anterior.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

En Windows el hash NTLM basta para autenticarte: no siempre hace falta crackear la password. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (nt1m-pth)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.